Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura
Escuela de Ingeniería Electrónica
Departamento de Sistemas e Informática

SISTEMAS EMBEBIDOS AVANZADOS II

Guía de instalación y primer proyecto

ESP-IDF + VS Code + FreeRTOS sobre ESP32-S3

Prof. Ing. Daniel Márquez
Octubre de 2026 — Versión 1.0

1. Introducción y objetivos

Esta guía describe, paso a paso, cómo instalar el entorno de desarrollo oficial de Espressif sobre Windows, crear un primer proyecto para el ESP32-S3 a partir de un ejemplo, grabarlo en la placa y verificar su funcionamiento. Además, se analiza el ejemplo desde el punto de vista de FreeRTOS, que es el sistema operativo de tiempo real que se estudia en la materia.

Al finalizar la guía, el estudiante debería ser capaz de:

- Instalar ESP-IDF con el **ESP-IDF Installation Manager (EIM)** y vincularlo con VS Code.
- Crear un proyecto a partir de un ejemplo, configurarlo para la placa de la cátedra, compilarlo, grabarlo y monitorear su salida.
- Interpretar la estructura de un proyecto ESP-IDF y su sistema de configuración (**menuconfig** / `sdkconfig`).
- Comprender que, en ESP-IDF, **FreeRTOS ya está corriendo** antes de que se ejecute el código del usuario, y que `app_main()` se ejecuta dentro de una **tarea**.

1.1. Componentes del entorno

Componente	Función
ESP-IDF	Framework oficial de Espressif (IoT Development Framework). Incluye drivers, bibliotecas, el sistema de compilación basado en CMake y una versión de FreeRTOS adaptada a los chips de Espressif, incluyendo soporte multinúcleo (SMP).
Toolchain	Compilador cruzado (GCC para la arquitectura Xtensa del ESP32-S3), depurador y utilidades asociadas.
EIM	ESP-IDF Installation Manager. Aplicación que descarga e instala ESP-IDF, el toolchain, Python y las demás herramientas necesarias, y verifica los prerrequisitos.
VS Code + extensión ESP-IDF	Entorno de edición. La extensión agrega comandos para crear proyectos, configurar, compilar, grabar, monitorear y depurar, pero **no instala ESP-IDF por sí sola**: necesita que el EIM lo haya instalado.
esptool	Programador que se comunica con el bootloader en ROM del chip para grabar la memoria flash.

2. Requisitos

2.1. Hardware

- Placa de desarrollo **ESP32-S3** con LED RGB integrado. La placa de la cátedra es del tipo **ESP32-S3 N16R8**: la sigla indica 16 MB de memoria flash (N16) y 8 MB de PSRAM (R8).
- Cable USB **de datos** (algunos cables solo transmiten energía y la PC no detecta la placa).

2.2. Software y PC

- Windows 10 u 11 de 64 bits.
- Conexión a Internet durante la instalación. Se descargan varios GB.
- Espacio libre en disco: se recomiendan al menos 10 GB.
- Permisos para instalar programas.

> **Importante:** ESP-IDF no tolera rutas con **espacios ni acentos**. Si el usuario de Windows tiene un nombre como `José Pérez`, los proyectos no deben crearse dentro de `C:\Users\José Pérez\`. Se recomienda usar una carpeta como `C:\esp\` para todos los proyectos de la materia.

3. Instalación del entorno

La instalación consta de tres partes, que deben realizarse en este orden: VS Code, la extensión ESP-IDF y, finalmente, ESP-IDF mediante el EIM. Se recomienda completarla **antes de la primera clase práctica**, ya que la descarga puede demorar entre 10 y 30 minutos según la conexión.

3.1. Instalar Visual Studio Code

1. Descargar VS Code desde **code.visualstudio.com** e instalarlo con las opciones por defecto.
2. Abrir VS Code para verificar que funciona.

3.2. Instalar la extensión ESP-IDF

1. En VS Code, abrir el panel de **Extensiones** (ícono de los cuatro cuadrados en la barra lateral izquierda, o `Ctrl+Shift+X`).
2. Buscar **ESP-IDF** e instalar la extensión publicada por **Espressif Systems**.
3. Al terminar, aparece en la barra lateral el ícono de Espressif. Al hacer clic se abre el panel **ESP-IDF: Explorer**, con una lista de comandos (**Commands**) y una sección **Advanced**.

> **Atención:** en este punto la extensión está instalada, pero **ESP-IDF todavía no**. Si se intenta crear un proyecto o se ejecuta **Select current ESP-IDF version**, la lista aparecerá vacía. Esto es normal: falta el paso siguiente.

3.3. Instalar ESP-IDF con el EIM

El **ESP-IDF Installation Manager (EIM)** es una aplicación independiente de VS Code. Si desde la extensión se ejecuta **Advanced > Open ESP-IDF Installation Manager** sin tenerlo instalado, se obtiene el mensaje **EIM not found**. Por eso se instala primero por separado.

1. Ingresar a la guía oficial de instalación para Windows: **docs.espressif.com/projects/esp-idf/en/stable/get-started/windows-setup.html**.
2. En la sección **Download the EIM**, descargar el instalador para **Windows (x64)** e instalarlo como cualquier otro programa. Como alternativa, la misma página explica cómo instalarlo con el gestor de paquetes **WinGet**.
3. Abrir la aplicación **ESP-IDF Installation Manager** desde el menú Inicio.
4. En la sección **Easy Installation**, hacer clic en **Start Easy Installation**. Esta opción instala la última versión estable de ESP-IDF con la configuración por defecto. El EIM verifica los prerrequisitos y ofrece instalar los que falten.
5. Esperar a que la instalación finalice y cerrar el EIM.
6. **Cerrar y volver a abrir VS Code**, para que la extensión detecte la nueva instalación.

3.4. Vincular ESP-IDF con VS Code

1. En el panel **ESP-IDF: Explorer > Commands**, hacer clic en **Select current ESP-IDF version**.
2. En la lista que se despliega en la parte superior de la ventana debe aparecer la versión instalada (por ejemplo, ESP-IDF v6.x). Seleccionarla.
3. Si la lista sigue vacía, repetir el punto 3.3 y verificar que la instalación haya terminado sin errores. El comando **Advanced > Doctor Command** genera un reporte de diagnóstico del entorno.

4. Conexión de la placa

La placa tiene dos conectores USB-C. Cualquiera de los dos permite grabar el programa y ver los mensajes por el monitor serie:

Conector	Descripción	Observaciones
USB	USB nativo del ESP32-S3 (controlador USB-Serial/JTAG interno).	No requiere drivers. Permite además **depurar** (breakpoints, ejecución paso a paso) sin hardware adicional.
UART / COM	Conversor USB-serie externo (CH343, CP210x u otro, según el fabricante).	En algunas PC requiere instalar el driver del fabricante del conversor.

Para identificar el puerto asignado, abrir el **Administrador de dispositivos** de Windows y desplegar **Puertos (COM y LPT)**. Al conectar la placa debe aparecer un nuevo puerto, por ejemplo `COM5`.

5. Primer proyecto: ejemplo blink

En lugar de escribir un proyecto desde cero, se parte del ejemplo oficial **blink**, incluido en ESP-IDF. Este ejemplo hace parpadear el LED de la placa y ya contempla el caso de un LED RGB direccionable, como el de nuestra placa.

5.1. Crear el proyecto

1. En el panel **ESP-IDF: Explorer**, ir a **Advanced > New Project Wizard**.
2. Se abre la pestaña **New Project**. En el buscador **Search Template By Name** escribir `blink`, o desplegar a mano **ESP-IDF Examples > get-started > blink**.
3. Seleccionar **blink**. A continuación aparece un formulario; completarlo según la tabla siguiente.

Campo	Valor	Comentario
Project Name	`blink`	Nombre de la carpeta del proyecto.
Enter Project directory	`C:\esp`	Cualquier ruta sin espacios ni acentos. La carpeta debe existir.
Choose ESP-IDF Target (IDF_TARGET)	`esp32s3`	Define el chip para el que se compila.
Choose ESP-IDF Board	ESP32-S3 chip (via builtin USB-JTAG)	Solo afecta la depuración (OpenOCD), no la grabación.
Choose serial port	El COM de la placa	Si se deja en **detect**, se elige luego desde la barra inferior.
Add your ESP-IDF Component directory	Vacío	Opcional. Sirve para agregar bibliotecas propias reutilizables entre proyectos.

4. Hacer clic en **Create Project** y aceptar cuando VS Code ofrezca abrir el proyecto.

5.2. ¿Cómo sabe el proyecto que el chip es un ESP32-S3?

El proyecto no lo detecta solo: lo define el **target**. Al elegir `esp32s3` en el formulario (o con el comando **Set Espressif Device Target**), la extensión ejecuta `idf.py set-target esp32s3`. Esto genera el archivo `sdkconfig` con `CONFIG_IDF_TARGET="esp32s3"`, y a partir de allí la compilación usa el toolchain, los drivers y el mapa de memoria del ESP32-S3.

Además, el ejemplo incluye archivos `sdkconfig.defaults.<chip>` con valores por defecto para cada chip. Al fijar el target se aplica el correspondiente, `sdkconfig.defaults.esp32s3`. Si no se define el target, el proyecto queda configurado para el **ESP32 original**, que es el valor por defecto.

Como protección adicional, al grabar, **esptool** lee la identificación del chip conectado. Si no coincide con el target, aborta la grabación con un mensaje del tipo *This chip is ESP32-S3, not ESP32*.

5.3. Ajustar el pin del LED (paso obligatorio para la placa N16R8)

El ejemplo trae como pin por defecto para el ESP32-S3 el **GPIO 38**, que corresponde a la placa de referencia de Espressif (ESP32-S3-DevKitC-1 v1.1). En la placa N16R8 de la cátedra, el LED RGB está conectado al **GPIO 48**. Si no se modifica, el programa funciona y envía mensajes por el puerto serie, pero el LED no responde: queda encendido en un color fijo o apagado.

1. En el panel **ESP-IDF: Explorer > Commands**, abrir **SDK Configuration Editor (menuconfig)**.
2. En el buscador escribir `Blink`. Aparece la sección **Example Configuration**.
3. Verificar o configurar los siguientes valores:

Opción	Valor	Significado
Blink LED type	LED strip	El LED es direccionable (WS2812), no un LED común.
LED strip backend peripheral	RMT	Periférico que genera la señal del LED.
Blink GPIO number	**48**	Pin al que está conectado el LED en la placa N16R8.
Blink period in ms	1000	Período de parpadeo.

4. Hacer clic en **Save**.

5.4. Compilar, grabar y monitorear

Antes de compilar, verificar en el panel **ESP-IDF: Explorer > Commands** (o en la barra inferior de VS Code) los siguientes parámetros:

- **Set Espressif Device Target:** `esp32s3`.
- **Select Port to Use:** el COM de la placa.
- **Select Flash Method: UART**. Funciona con cualquiera de los dos conectores.

Luego ejecutar **Build, Flash and Monitor**, que compila, graba y abre el monitor serie en un solo paso. La primera compilación demora varios minutos, porque se compila todo el framework y se descarga el componente `led_strip`. Las siguientes son mucho más rápidas, ya que solo se recompila lo modificado.

Si todo funciona, el LED parpadea en blanco una vez por segundo y en el monitor se observan mensajes similares a los siguientes:

```
I (281) main_task: Started on CPU0
I (291) main_task: Calling app_main()
I (291) example: Example configured to blink addressable LED!
I (301) example: Turning the LED OFF!
I (1301) example: Turning the LED ON!
I (2301) example: Turning the LED OFF!
```

La letra `I` indica un mensaje de nivel *Info*, el número entre paréntesis es el tiempo en milisegundos desde el arranque y luego aparece la etiqueta (*tag*) del módulo que generó el mensaje. Para salir del monitor se usa `Ctrl+]`.

> **Observación:** las dos primeras líneas no las escribe el ejemplo, sino ESP-IDF. Muestran que existe una tarea llamada **main_task**, que corre en el núcleo 0 y que es la que llama a `app_main()`. Este punto se desarrolla en la sección 8.

6. Estructura del proyecto

```
blink/
├── CMakeLists.txt              Proyecto: nombre y sistema de compilación
├── sdkconfig                   Configuración efectiva (generada)
├── sdkconfig.defaults          Valores por defecto comunes
├── sdkconfig.defaults.esp32s3  Valores por defecto para el ESP32-S3
├── sdkconfig.defaults.<chip>   ... uno por cada chip soportado
├── main/
│   ├── CMakeLists.txt          Registra los fuentes del componente main
│   ├── Kconfig.projbuild       Define las opciones de 'Example Configuration'
│   ├── idf_component.yml       Dependencias externas (led_strip)
│   └── blink_example_main.c    Código de la aplicación
├── managed_components/         Componentes descargados (generada)
└── build/                      Resultados de compilación (generada)
```

Algunos puntos a destacar:

- Un proyecto ESP-IDF está formado por **componentes**. `main` es el componente de la aplicación; ESP-IDF aporta muchos más (drivers, FreeRTOS, log, etc.).
- El archivo `idf_component.yml` declara que el proyecto necesita el componente `espressif/led_strip`. El **gestor de componentes** lo descarga automáticamente del registro de Espressif en la primera compilación y lo guarda en `managed_components/`.
- El archivo `Kconfig.projbuild` define las opciones propias del ejemplo, que aparecen en menuconfig dentro de **Example Configuration**.
- Las carpetas `build/` y `managed_components/` y el archivo `sdkconfig` se regeneran; no conviene editarlos a mano.

6.1. El sistema de configuración: menuconfig y sdkconfig

Cada opción configurable de ESP-IDF tiene un nombre que comienza con `CONFIG_`. El archivo `sdkconfig` guarda el valor de todas las opciones del proyecto. Por ejemplo, la sección del ejemplo luce así:

```
# CONFIG_BLINK_LED_GPIO is not set
CONFIG_BLINK_LED_STRIP=y
CONFIG_BLINK_LED_STRIP_BACKEND_RMT=y
# CONFIG_BLINK_LED_STRIP_BACKEND_SPI is not set
CONFIG_BLINK_GPIO=48
CONFIG_BLINK_PERIOD=1000
```

- `=y` indica una opción booleana activada. Una opción desactivada se escribe como comentario: `# ... is not set`.

- Las opciones numéricas se guardan con su valor, como el pin o el período.
- Durante la compilación, cada opción se convierte en una **macro de C** disponible en el código. Por eso el ejemplo puede escribir `#define BLINK_GPIO CONFIG_BLINK_GPIO` sin tener el número de pin escrito a mano.

> **Buena práctica:** modificar la configuración siempre desde **menuconfig**, que resuelve las dependencias entre opciones. Editar `sdkconfig` a mano puede dejar configuraciones inconsistentes.

7. El LED RGB direccionable (WS2812) y el periférico RMT

El LED de la placa no es un LED común que se enciende al poner un pin en alto. Es un **WS2812**: un encapsulado que contiene tres LEDs (rojo, verde y azul) y un pequeño **controlador** que recibe el color por un único cable de datos, mediante un protocolo serie.

7.1. Protocolo

- El color se envía como una trama de **24 bits**: 8 bits de verde, 8 de rojo y 8 de azul, en ese orden (formato **GRB**).
- Cada bit se codifica por el **ancho del pulso** en alto, no por el nivel. Un pulso corto (del orden de 0,4 µs) representa un 0 y uno largo (del orden de 0,8 µs) representa un 1, con un período de bit de aproximadamente 1,25 µs.
- Tras los 24 bits, una pausa en bajo de varias decenas de microsegundos (*reset* o *latch*) indica al controlador que aplique el nuevo color.
- El controlador **mantiene el último color recibido** y genera internamente el PWM de cada LED. Si nunca recibe una trama válida, puede quedar en un color arbitrario: es lo que ocurre si el pin configurado es incorrecto.
- Cada WS2812 tiene una salida de datos que permite **encadenar** muchos LEDs en el mismo cable: cada uno toma sus 24 bits y retransmite el resto. Así funcionan las tiras LED, y por eso el componente se llama `led_strip`, aunque la placa tenga un solo LED.



7.2. ¿Por qué no se maneja directamente con el GPIO?

Eléctricamente, el pin 48 es una salida digital. Sin embargo, generar por software pulsos de fracciones de microsegundo con precisión (*bit-banging*) es frágil: cualquier interrupción o cambio de contexto del sistema operativo en medio de la trama la corrompe. Por eso se usa el periférico **RMT** (*Remote Control Transceiver*), diseñado originalmente para controles remotos infrarrojos, que genera formas de onda definidas como secuencias de pares (nivel, duración) **por hardware**, sin intervención de la CPU.

El ESP32-S3 dispone de una **matriz de GPIO** que permite conectar la salida de casi cualquier periférico interno a casi cualquier pin. En este caso, la salida del RMT se conecta al GPIO 48. La alternativa **SPI** que ofrece menuconfig es otra técnica con el mismo fin: cada bit del WS2812 se codifica como un patrón de varios bits SPI y se usa solo la línea MOSI.

> **Relación con la materia:** este es un ejemplo concreto de por qué, en un sistema con RTOS, conviene delegar en periféricos las tareas con requisitos de temporización estrictos. La CPU queda libre para que el planificador ejecute otras tareas.

8. FreeRTOS en el ejemplo: app_main() es una tarea

Aunque el código del ejemplo no crea ninguna tarea ni inicia el planificador, **el programa ya está corriendo sobre FreeRTOS**. En ESP-IDF, FreeRTOS no es una biblioteca opcional: forma parte del framework y se inicia automáticamente antes de que se ejecute cualquier código del usuario.

8.1. Secuencia de arranque

Según la documentación de Espressif (*Application Startup Flow*), al energizar o reiniciar la placa ocurre lo siguiente:

1. **Bootloader de primera etapa (ROM):** código grabado en la ROM del chip, que no se puede modificar. Lee los pines de *strapping* para decidir el modo de arranque y carga desde la flash el bootloader de segunda etapa.
2. **Bootloader de segunda etapa:** lee la tabla de particiones, carga la aplicación en RAM o la mapea desde la flash, verifica su integridad y salta a su punto de entrada.
3. **Inicialización de la aplicación:** configura la CPU, los relojes, la memoria, el heap, la consola y los componentes; arranca el segundo núcleo.
4. **Creación de la tarea principal e inicio del planificador:** se crea la tarea **main_task** y se pone en marcha el planificador de FreeRTOS en ambos núcleos.
5. **Llamada a app_main():** la tarea **main_task** ejecuta la función `app_main()` del usuario.

8.2. Consecuencias para el programador

Aspecto	En ESP-IDF
Punto de entrada	No existe `main()`. El programador escribe `void app_main(void)`.
Planificador	Ya está corriendo cuando se ejecuta `app_main()`. **No se llama** a `vTaskStartScheduler()`.
Contexto de app_main()	Se ejecuta dentro de una tarea FreeRTOS (**main_task**), con todas sus propiedades: prioridad, stack propio, estados (lista, bloqueada, en ejecución).
Prioridad de main_task	Fija: una por encima de la mínima, es decir, **1**.
Stack de main_task	Configurable con `CONFIG_ESP_MAIN_TASK_STACK_SIZE`, en **bytes**.
Núcleo	Configurable con `CONFIG_ESP_MAIN_TASK_AFFINITY`. Por defecto, **CPU0**, como se ve en el log: *main_task: Started on CPU0*.
Retorno de app_main()	Está **permitido**. La tarea main_task se elimina y el sistema sigue funcionando con las demás tareas. Una tarea FreeRTOS común, en cambio, nunca debe retornar.

Por lo tanto, existen dos estilos válidos de programación:

- Usar `app_main()` **como una tarea más**, con su propio lazo infinito. Es lo que hace el ejemplo blink.
- Usar `app_main()` **solo para inicializar**: configurar periféricos, crear las tareas de la aplicación con `xTaskCreate()` y retornar. Es el estilo habitual en aplicaciones con varias tareas, y se trabajará en el próximo ejemplo.

8.3. Análisis del código

La función principal del ejemplo es la siguiente:

```
void app_main(void)
{
    /* Configure the peripheral according to the LED type */
    configure_led();

    while (1) {
        ESP_LOGI(TAG, "Turning the LED %s!", s_led_state == true ? "ON" : "OFF");
        blink_led();
        /* Toggle the LED state */
        s_led_state = !s_led_state;
        vTaskDelay(CONFIG_BLINK_PERIOD / portTICK_PERIOD_MS);
    }
}
```

Línea por línea:

- `configure_led();` inicializa el driver `led_strip` sobre el periférico RMT y el GPIO configurado en menuconfig. Se ejecuta una sola vez, dentro de la tarea main_task.
- `while (1)` es el lazo infinito típico del cuerpo de una tarea FreeRTOS.
- `ESP_LOGI(...)` imprime un mensaje de nivel *Info* por la consola, con la marca de tiempo y la etiqueta `TAG`. La expresión `s_led_state == true ? "ON" : "OFF"` elige el texto según el estado actual.
- `blink_led();` enciende el LED en blanco tenue (16, 16, 16 sobre 255 en cada color) o lo apaga, según `s_led_state`. Internamente llama a `led_strip_set_pixel()`, que escribe un buffer en memoria, y a `led_strip_refresh()`, que transmite ese buffer al LED mediante el RMT.
- `s_led_state = !s_led_state;` invierte el estado para la próxima iteración.
- `vTaskDelay(...)` es la línea clave desde el punto de vista del RTOS. **Bloquea** la tarea durante la cantidad de *ticks* indicada. Mientras está bloqueada, la tarea no consume CPU y el planificador ejecuta otras tareas o, si no hay ninguna lista, la tarea **IDLE** de ese núcleo.

8.4. El tick del sistema

`vTaskDelay()` recibe su argumento en *ticks*, no en milisegundos. La frecuencia del tick se configura con `CONFIG_FREERTOS_HZ`, cuyo valor por defecto en ESP-IDF es **100 Hz**, es decir, un tick cada 10 ms. La constante `portTICK_PERIOD_MS` vale `1000 / CONFIG_FREERTOS_HZ`, por lo que la expresión del ejemplo resulta:

```
CONFIG_BLINK_PERIOD / portTICK_PERIOD_MS  =  1000 / 10  =  100 ticks
```

Una forma equivalente y más usual es `vTaskDelay(pdMS_TO_TICKS(1000))`. En ambos casos, la resolución temporal está limitada por el tick: con 100 Hz no es posible un retardo de 5 ms, ya que se redondea a un número entero de ticks.

8.5. Otras tareas que ya están corriendo

Además de main_task, ESP-IDF crea sus propias tareas de sistema. Entre ellas:

- **IDLE0 e IDLE1:** una tarea ociosa por núcleo, que se ejecuta cuando no hay otra tarea lista.
- **ipc0 e ipc1:** permiten que un núcleo solicite la ejecución de una función en el otro.
- **esp_timer:** atiende las funciones de los temporizadores de alta resolución de ESP-IDF.
- **Tmr Svc:** la tarea de servicio de los *software timers* de FreeRTOS.

8.6. Diferencias con FreeRTOS estándar

Para quienes conocen FreeRTOS en microcontroladores de un solo núcleo, conviene tener presentes estas diferencias:

Aspecto	FreeRTOS estándar	FreeRTOS en ESP-IDF
Inicio	`main()` crea tareas y llama a `vTaskStartScheduler()`.	El planificador ya está iniciado; el usuario escribe `app_main()`.
Tamaño del stack en xTaskCreate()	En **palabras** (StackType_t).	En **bytes**.
Núcleos	Uno.	Dos (SMP). Existe `xTaskCreatePinnedToCore()` para fijar una tarea a un núcleo.
Secciones críticas	`taskENTER_CRITICAL()` deshabilita interrupciones.	`taskENTER_CRITICAL(&spinlock)`: además usa un *spinlock* para excluir al otro núcleo.
Watchdog	Opcional, a cargo del usuario.	El *Task Watchdog* vigila las tareas IDLE: una tarea que nunca se bloquea dispara un aviso.

9. Problemas frecuentes

Síntoma	Causa probable y solución
EIM not found al usar la opción del EIM desde VS Code	El EIM no está instalado. Instalarlo por separado, según la sección 3.3.
Select current ESP-IDF version no muestra ninguna versión	ESP-IDF no está instalado, o VS Code no se reinició después de instalarlo. Completar la sección 3.3 y reiniciar VS Code.
El New Project Wizard no muestra plantillas ni ejemplos	Igual que el caso anterior: falta ESP-IDF o no está seleccionada la versión.
Errores de compilación con rutas extrañas	El proyecto está en una ruta con espacios o acentos. Moverlo, por ejemplo, a `C:\esp\`.
La PC no detecta la placa	Cable de solo carga, o falta el driver del conversor USB-serie (conector UART/COM). Probar otro cable o el conector USB nativo.
Error **Access denied** sobre el puerto COM	Otro programa tiene el puerto abierto (Arduino IDE, PuTTY, otro monitor). Cerrarlo.
No logra grabar (*Failed to connect*)	Forzar el modo de descarga: mantener presionado **BOOT**, presionar y soltar **RST** y luego soltar **BOOT**. Reintentar.
Llegan mensajes por el monitor, pero el LED queda fijo en un color o apagado	Pin incorrecto. En la placa N16R8 debe configurarse **GPIO 48** en menuconfig (sección 5.3).
Los colores aparecen intercambiados (rojo y verde)	Algunas placas usan un LED con otro orden de color. Se ajusta en la configuración del driver `led_strip`.
Mensaje del *Task Watchdog* en el monitor	Una tarea ejecuta un lazo sin bloquearse nunca (sin `vTaskDelay()` ni espera de cola o semáforo) e impide que corra la tarea IDLE.

10. Actividades propuestas

1. Modificar el período de parpadeo a 250 ms **desde menuconfig**, sin tocar el código. Verificar el cambio en las marcas de tiempo del monitor.
2. Modificar `blink_led()` para que el LED encienda en rojo, verde o azul en lugar de blanco. Recordar el orden de los argumentos de `led_strip_set_pixel()`: índice del LED, rojo, verde y azul.
3. Reemplazar la expresión del retardo por `vTaskDelay(pdMS_TO_TICKS(CONFIG_BLINK_PERIOD))` y justificar por qué el comportamiento es el mismo.
4. Configurar un período de 15 ms y medir, a partir de las marcas de tiempo del log, el período real. Explicar la diferencia en función del tick del sistema.
5. Reemplazar `vTaskDelay()` por un lazo de espera activa (por ejemplo, un `for` vacío muy largo). Observar qué informa el monitor y explicar el resultado.
6. Agregar al final de `app_main()`, fuera del `while`, un mensaje con `ESP_LOGI`. ¿Se imprime alguna vez? ¿Por qué? ¿Qué ocurriría si se eliminara el `while (1)`?
7. Modificar el ejemplo para que, una tarea haga parpadear el Led cada 500 ms y otra envíe un mensaje por consola cada 1 s con una cuenta que llegue hasta 999 segundos y sea cíclica. Crear una tarea para cada una de las acciones indicadas y que corran fuera de la tarea del void app_main(void).

11. Referencias

- Espressif Systems. *ESP-IDF Programming Guide — Installation of ESP-IDF and Tools on Windows*. docs.espressif.com/projects/esp-idf/en/stable/get-started/windows-setup.html
- Espressif Systems. *ESP-IDF Installation Manager (EIM)*. docs.espressif.com/projects/idf-im-ui/en/latest/
- Espressif Systems. *ESP-IDF Programming Guide — Application Startup Flow (ESP32-S3)*. docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/startup.html
- Espressif Systems. *ESP-IDF Programming Guide — FreeRTOS (IDF)*. docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/system/freertos_idf.html
- Espressif Systems. Ejemplo *get-started/blink*. github.com/espressif/esp-idf/tree/master/examples/get-started/blink
- Espressif Systems. Componente *led_strip*. components.espressif.com/components/espressif/led_strip
- Worldsemi. *WS2812B Intelligent control LED integrated light source — Datasheet*.